



同濟大學
TONGJI UNIVERSITY

课程论文

**CineScope：面向电影数据的
分析与推荐系统设计与实现**

课程名称：数据挖掘与数据分析

组 号：第 17 组

小组成员：王雷（2351299）、黄景胤（2351129）、
周达（2354185）、林琪（2352609）

指导老师：赵钦佩

二〇二六年六月

摘要

随着在线流媒体平台的普及，电影数据呈现出规模庞大、结构异构、关联复杂等特点。如何从多源异构数据中完成清洗融合，并进一步支撑检索、可视化、票房预测与个性化推荐，是数据挖掘领域具有代表性的工程问题。针对这些问题，我们设计并实现了一套面向电影数据的分析与推荐系统 CineScope。

系统以 MovieLens、TMDb 与 IMDb 等多源数据为基础，构建了包含数据抽取、转换与加载（ETL）、探索性可视化、票房回归建模、内容推荐与协同过滤推荐、统一后端聚合以及前端交互展示的完整技术链路。在数据层，项目最终产出 9727 部电影、100811 条用户评分与 3678 条用户标签；在模型层，采用 XGBoost 与 LightGBM 加权融合完成票房预测，在验证集上取得 $R_{\log}^2 \approx 0.65$ ；在推荐层，分别基于字符级 TF-IDF/词袋相似度与 Item/User KNN 实现内容推荐和协同过滤推荐，并通过独立微服务对外提供 HTTP 接口；在应用层，基于 React 与 FastAPI 实现电影搜索、统计可视化、RAG 问答与 Agent 推荐等功能。

实验结果表明，CineScope 能够在统一架构下有效整合数据处理、机器学习推理与交互式服务，形成可复现、可扩展的电影数据分析工程范式。特征重要性与相关性分析进一步表明，预算与热度是票房预测的主导信号；内容推荐与协同过滤则在同系列召回与用户行为召回之间形成互补。本文还对系统局限性进行了讨论，并提出了后续在推荐评估、混合推荐与语义检索方面的改进方向。

关键词：电影数据挖掘；推荐系统；票房预测；数据可视化

第 1 章 绪论

1.1 研究背景

电影是数字娱乐领域最具代表性的内容形态之一。近年来，流媒体平台、在线评分社区与开放电影数据库快速发展，电影相关数据在规模与维度上持续增长：用户评分与标签等行为数据能够反映受众偏好，影片类型、剧情简介与语言等元数据支撑内容理解，预算、票房与热度等商业指标则为市场分析提供重要依据。这些多源数据为电影检索、统计分析和智能推荐等应用提供了丰富素材。

然而，真实电影数据往往并不规整。不同数据源在字段命名、主键体系和缺失模式上存在显著差异，预算与票房等关键商业字段在公开数据集中常常不完整；用户评分矩阵又具有高度稀疏性，给协同过滤建模带来困难。与此同时，数据处理、模型训练与系统开发若彼此割裂，就难以形成从数据清洗到交互式服务落地的完整链路。因此，有必要围绕电影多源数据构建一套结构清晰、模块边界明确、可复现可扩展的分析与推荐系统。

基于上述背景，本文设计并实现 CineScope 系统。该系统以 MovieLens、TMDb 与 IMDb 数据为基础，贯通数据清洗、探索分析、建模预测、推荐服务与前端交互，旨在为电影数据分析提供一体化的工程化解决方案。

1.2 问题定义

围绕电影数据分析与推荐这一应用场景，本文将研究问题概括为以下四个方面。

(1) 多源数据融合问题。系统需要从 MovieLens 获取用户行为数据，并结合 TMDb/IMDb 补充影片详情与财务元数据。如何在保证样本规模的前提下完成主键对齐、重复记录处理、缺失字段控制与质量审计，形成可供建模和在线服务使用的终版数据集，是首要研究问题。

(2) 票房建模问题。电影票房具有明显右偏分布，且同时受到预算、类型、评分统计、语言等多类特征影响。在有限样本条件下，如何选择合适的回归目标变换与模型结构，并通过可复现的离线训练流程获得稳定、可比较的评估结果，是本文关注的第二个问题。

(3) 推荐生成问题。实际应用中，用户既可能基于某部已看电影寻找相似影片，也可能希望根据个人历史评分获得个性化推荐。如何分别实现内容相似推荐

与协同过滤推荐，并在工程上将推荐能力独立为可调用服务，同时处理冷启动与候选不足等场景，是推荐模块设计的核心问题。

(4) 系统交付问题。除算法模型外，电影数据分析能力还需要通过可交互的应用系统呈现。如何将电影检索、统计可视化、推荐代理、RAG 检索以及 Agent 交互整合为前后端分离、服务边界清晰的原型，是工程实现层面必须解决的问题。

1.3 研究目标

围绕上述问题，本文以 CineScope 为研究对象，设定如下研究目标：

1. **构建可复现的数据处理链路：**完成 MovieLens、TMDb 与 IMDb 数据的抽取、清洗与融合，产出包含电影主表、评分表、标签表、类型统计及质量报告在内的终版数据集，为后续分析与建模提供统一数据基础。
2. **实现票房预测模型：**基于非零票房样本构建特征工程与训练流程，采用 XGBoost 与 LightGBM 集成方法完成票房回归建模，并在验证集上给出可比较的评估指标。
3. **实现双通道推荐服务：**分别构建内容推荐与协同过滤推荐能力，通过独立微服务对外提供 HTTP 接口，支持相似电影召回与用户个性化推荐。
4. **完成可交互系统原型：**基于 React 与 FastAPI 实现电影搜索、统计可视化、推荐展示及 RAG/Agent 扩展能力，验证“数据层—模型层—服务层—展示层”分层架构的工程可行性。

通过实现上述目标，本文期望形成一套可运行、可验证、可扩展的电影数据分析系统。

第 2 章 系统设计与关键技术

2.1 系统总体架构

CineScope 采用“数据层—模型层—服务层—展示层”四层架构。数据层负责多源数据清洗与终版数据产出；模型层包含票房预测产物与推荐离线产物；服务层由统一后端与独立推荐微服务组成；展示层基于 React 提供搜索、推荐与可视化交互。系统总体架构如图 2.1 所示。

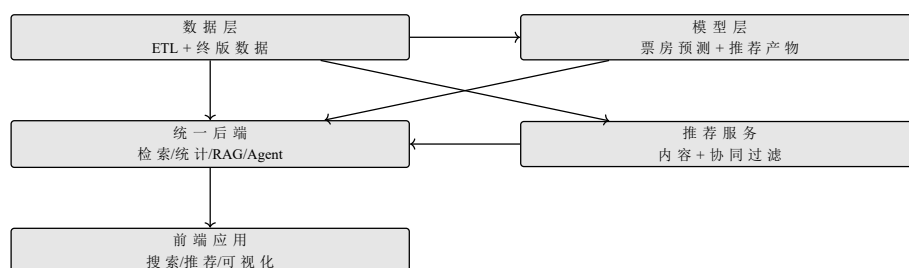


图 2.1 CineScope 系统总体架构

架构设计遵循三项原则：（1）训练与推理分离，离线训练脚本将模型产物写入 `models/artifacts/`，在线服务仅加载序列化文件；（2）推荐服务独立部署，避免业务后端与推荐算法强耦合；（3）可视化与推理解耦，探索性分析模块只读终版数据，不参与在线请求链路。

2.2 数据处理与特征构建

2.2.1 ETL 流水线

ETL 模块位于 `etl/cinescope_data/`，核心流程包括：

1. 下载并解压 MovieLens 数据集（默认 `ml-latest-small`）；
2. 清洗电影、评分、标签与链接表，统一主键为 `movie_id`；
3. 调用 TMDb API 拉取影片详情，并以 IMDb ID 作为财务元数据的回退匹配键；
4. 融合用户评分统计、标签聚合、影片文本与财务字段，输出终版 CSV/JSON 文件；
5. 生成 `quality_report.json` 记录丢弃规则、缺失率与提交范围。

在提交范围内，系统保留 `movies.csv`、`ratings.csv`、`tags.csv` 与 `genre_stats.csv` 四类文件。电影主表包含 20 个字段，覆盖标题、年份、类

型、评分统计、简介、片长、热度、预算、票房、语言与海报链接等。表 2.1 列出了电影主表的核心字段及其在系统中的用途。

表 2.1 电影主表核心字段说明

字段	含义	主要用途
movie_id	MovieLens 电影编号	全局主键、关联评分与标签
title_clean	规范化片名	搜索、推荐种子匹配
genres_json	电影类型列表	筛选、特征编码、内容推荐
rating_mean	平均评分	排序、统计分析、热门兜底
overview	剧情简介	文本相似度与统一搜索
budget / revenue	预算与票房	可视化、票房预测
tmdb_popularity	TMDb 热度	特征工程、相关性分析

2.2.2 数据质量约束

为保证下游建模稳定性，ETL 实施如下约束：

- 仅保留成功匹配 TMDb 详情的电影记录，最终电影数为 9727 部；
- 对无 TMDb ID 的电影相关评分与标签予以剔除；
- 对预算/票房字段执行外部财务元数据注入，非零票房样本 6235 条；
- 输出数据集摘要与质量报告，支持 RAG 模块进行项目级问答。

2.3 票房预测方法

2.3.1 目标变量与样本筛选

票房分布呈明显右偏，直接以原始美元金额回归会导致大票房样本主导损失函数。因此，训练目标设为

$$y = \log(1 + \text{revenue}), \quad (2.1)$$

推理阶段通过 $\exp(y) - 1$ 还原至原始票房空间。训练样本仅保留 $\text{revenue} > 0$ 的电影，共 6235 条，按 8:2 划分训练集（4988）与验证集（1247）。

2.3.2 特征工程

特征工程由 `models/revenue_features.py` 统一管理，共 66 维，包括：

- **数值特征（14 维）**：年份、片长、TMDb 热度、预算、评分计数、评分均值/中位数、标签数等；

- 类别特征：原始语言 one-hot 编码；
- 类型特征：电影类型 one-hot 编码（由 genres_json 解析）。

缺失值填充与类别取值在训练阶段拟合，并序列化至 feature_schema.json，保证训练与推理一致。

2.3.3 集成回归模型

项目同时训练 XGBoost 与 LightGBM 两个梯度提升回归器，并在验证集上搜索最优加权比例。设两模型在 log 空间输出分别为 $\hat{y}_1, \hat{y}_2$ ，则集成预测为

$$\hat{y} = w_1 \hat{y}_1 + w_2 \hat{y}_2, \quad w_1 + w_2 = 1. \quad (2.2)$$

权重选择以验证集 $\text{RMSE}_{\log}$ 最小为准，最终得到 $w_{\text{XGB}} = 0.88$ 、 $w_{\text{LGB}} = 0.12$ 。模型产物写入 models/artifacts/revenue_prediction/，推理入口为 models/revenue_predictor.py。

2.3.4 训练超参数

两模型均采用梯度提升回归框架，并在验证集上启用早停。XGBoost 设置学习率 $\eta = 0.035$ 、最大树深 4、子采样率 0.85，最大迭代轮数 900、早停轮数 60，实际最优迭代为 513 轮；LightGBM 设置学习率 0.035、叶子数 31、最小叶子样本 12，同样采用 900 轮上限与 60 轮早停。该配置在控制过拟合的同时，为右偏票房目标提供了较稳定的非线性拟合能力。

2.3.5 特征设计说明

除原始数值字段外，特征工程还对预算、热度与评分计数执行 $\log(1+x)$ 变换，并构造 release_age（以 2018 年为参考年份）刻画影片新旧程度。类别侧，原始语言与 19 类电影类型分别进行 one-hot 编码；缺失值填充策略与类别取值空间在训练阶段拟合后写入 feature_schema.json，保证离线训练与后续推理使用同一套变换规则。该设计使模型能够同时利用商业指标、受众反馈与内容属性三类信息。

2.4 推荐系统方法

推荐模块实现为独立 FastAPI 微服务，包含内容推荐与协同过滤推荐两条路径。

2.4.1 内容推荐

给定种子电影 m ，系统构造两路文本表示：

1. **简介通道**：将 `title_clean` 与 `overview` 拼接后，使用字符级 TF-IDF（2–4 gram）向量化，计算余弦相似度；
2. **内容通道**：将标题、类型、热门标签与语言拼接为 `content soup`，使用 `CountVectorizer` 向量化后计算余弦相似度。

两路相似度按权重融合（默认 0.55/0.45），排除种子电影自身后取 Top- K 。该方法适合“看过 m ，寻找内容相近电影”的场景。

2.4.2 协同过滤推荐

基于 610 名用户对 9727 部电影的 100811 条评分，构建稀疏评分矩阵，并实现：

- **Item KNN**：以电影在全体用户上的评分向量表示影片，召回相似电影；
- **User KNN**：以用户在全体电影上的评分向量表示用户，生成个性化候选。

当同时给定 `user_id` 与 `movie_name` 时，系统按 0.4/0.6 权重融合 Item 与 User 分数；仅给定用户时返回纯个性化列表。若近邻候选不足，则使用基于评分计数与均值的贝叶斯热门分数兜底。

2.4.3 离线产物构建

推荐服务不直接在线训练，而是由 `recommender/scripts/build_all.py` 离线生成 `vectorizer`、稀疏矩阵与 KNN 模型，存放于 `recommender/artifacts/`。该设计保证服务启动时仅需加载产物，降低在线延迟。

2.5 应用层与智能问答

2.5.1 统一后端

统一后端（`server/`）聚合以下能力：

- **电影检索**：支持标题/简介/类型/标签统一搜索，并按类型、语言、评分过滤，同时提供分页、排序与详情查询；
- **统计分析**：输出数据集摘要、类型分布、预算趋势、票房散点与相关性矩阵；

- **推荐代理**：将前端请求转发至推荐微服务，并封装统一响应格式；
- **RAG 检索**：对数据集摘要、质量报告与项目文档进行轻量内存检索；
- **Agent 推荐**：支持自然语言偏好描述，结合 RAG 上下文生成推荐解释。

2.5.2 RAG 与 Agent 工作流

RAG 模块采用轻量内存检索，索引对象包括数据集摘要、质量报告、类型统计、项目说明文档，以及由 `movies.csv` 抽取的电影画像与排行榜摘要。索引总规模约 3686 条文档，其中电影画像与排行榜文档 3680 条（覆盖评分计数不少于 10 的影片及热度 Top 3000），其余为数据集级元信息。检索阶段综合标题完全匹配、正文子串命中与 `token` 重叠打分，并对统计类查询提升 `dataset_summary`、`quality_report` 等文档权重，最终返回 Top-*K* 上下文供下游模块使用。

Agent 模块区分项目问答与电影推荐两类意图：前者经 RAG 检索后组织自然语言回答；后者先由规则解析偏好（类型、语言、评分阈值与排除项），再按场景路由至内容推荐、协同过滤或本地筛选路径，并结合推荐服务生成候选与解释。该设计将事实检索与候选生成解耦，使统计数字与电影 ID 由本地服务决定，语言模型主要负责组织可读回答。

2.5.3 前端交互

前端基于 React + TypeScript + Vite 实现三类页面：

- **Search**：电影搜索、筛选、排序与详情面板；
- **Recommend**：内容推荐、协同过滤推荐与 Agent 推荐；
- **Atlas**：数据集概览、类型分布、预算趋势、气泡图与相关性视图。

表 2.2 汇总了统一后端主要 REST 接口。

表 2.2 统一后端主要 API 接口

方法	路径	功能
GET	/movies	电影搜索、筛选、分页与排序
GET	/movies/{id}	单部电影详情
POST	/recommendations	内容、协同与 Agent 推荐
GET	/stats/summary	数据集概览统计
GET	/stats/genres	电影类型分布
POST	/rag/search	项目文档与数据检索
POST	/agent/chat	项目问答与意图识别
POST	/agent/recommend	自然语言电影推荐

第 3 章 实验设计与结果分析

3.1 实验环境与数据集

3.1.1 软硬件环境

实验在本地环境完成，主要依赖如下：

- **Python 环境：**pandas、numpy、scikit-learn、scipy、XGBoost、LightGBM、FastAPI、uvicorn；
- **前端环境：**Node.js、React 19、TypeScript、Vite；
- **服务端口：**前端 5173、统一后端 8000、推荐服务 3000。

3.1.2 终版数据集统计

表 3.1 给出了 CineScope 终版数据集规模。该数据集源自 MovieLens ml-latest-small，并与 TMDb 详情完成全量匹配。

表 3.1 CineScope 终版数据集统计

指标	数值
电影数量	9727
用户数量	610
评分数量	100811
标签数量	3678
电影类型数	19
平均评分	3.50
非零票房样本	6235
非零预算样本	5844
预算与票房均非零样本	5230

评分矩阵稀疏度约为 $1 - 100811 / (610 \times 9727) \approx 98.3\%$ 。类型分布呈明显长尾，Drama 与 Comedy 占据多数，电影数量排名前列的类型见表 3.2。

表 3.2 电影类型分布 (Top 10)

排名	电影类型	电影数量 (部)
1	Drama	4355
2	Comedy	3753
3	Thriller	1891
4	Action	1827
5	Romance	1594
6	Adventure	1261
7	Crime	1195
8	Horror	976
9	Sci-Fi	975
10	Fantasy	779

3.2 数据清洗实验结果

ETL 实验重点验证可用样本规模与字段完整性。质量报告显示：

- 原始电影 9742 部，剔除无 TMDb 详情记录后保留 9727 部；
- 全部终版电影均具有非空 `tmdb_id` 与 TMDb 详情；
- 财务元数据匹配成功 9386 部，其中 IMDb 回退匹配 4 部；
- 简介缺失 35 部、片长缺失 41 部，整体缺失率较低。

从剔除规则看，无 `tmdb_id` 记录 8 部、无 TMDb 详情记录 7 部，合计丢弃 15 部影片；与之关联的评分与标签分别减少 25 条与 5 条，对整体行为数据影响有限。财务元数据方面，TMDb 电影端点匹配 9628 部、电视剧端点匹配 99 部，另有 35 部通过 IMDb ID 回退完成详情补全；最终 9386 部影片获得预算或票房字段，其中预算与票房均非零的交集样本为 5230 条，可直接用于预算—票房相关性分析与散点可视化。

上述结果表明，数据清洗策略能够在保证样本规模的同时控制关键字段缺失，为票房预测与内容推荐提供稳定输入。

3.3 票房预测实验

3.3.1 实验设置

票房预测实验采用第 2.3 节方法，随机种子为 42，验证集比例 0.2。评价指标包括 log 空间 RMSE、MAE、 R^2 ，以及还原到美元空间后的 RMSE、MAE 与 MAPE。

3.3.2 单模型与集成结果

表 3.3 汇总了 XGBoost、LightGBM 与集成模型在验证集上的表现。可以看到，两模型在 log 空间性能接近，集成后 $\text{RMSE}_{\log}$ 基本持平并略降至 1.275， $R^2_{\log}$ 提升至 0.652。

表 3.3 票房预测模型验证集指标

模型	$\text{RMSE}_{\log}$	$\text{MAE}_{\log}$	$R^2_{\log}$
XGBoost	1.275	0.858	0.652
LightGBM	1.296	0.868	0.641
集成模型	1.275	0.857	0.652

在原始票房空间，集成模型验证集 $\text{RMSE} \approx 9.53 \times 10^7$ 美元， $\text{MAE} \approx 3.94 \times 10^7$ 美元。需要指出的是，由于票房分布极端偏斜，MAPE 对低票房样本非常敏感，因此该指标在本任务中仅供参考。

3.3.3 结果分析

$R^2_{\log} \approx 0.65$ 说明模型能够解释约 65% 的 log 票房方差，在当前数据集与特征设定下具有较好的回归拟合效果。进一步比较单模型与集成效果可见，XGBoost 在 log 空间 $R^2_{\log}$ 已达 0.652，LightGBM 为 0.641；集成权重 88:12 的设定表明当前特征体系下 XGBoost 贡献更大，集成主要起到误差微调 and 稳定性增强作用。

3.3.4 特征重要性分析

表 3.4 给出了 XGBoost 模型的特征重要性 Top 10（按训练损失下降量统计）。预算 (budget) 与 TMDb 热度 (tmdb_popularity) 位列前两位，合计贡献超过一半；上映年份、评分计数及其对数变换紧随其后，反映影片新旧程度与受众基础对票房的间接影响。语言特征 lang_en 进入前十，说明英语发行市场对

票房仍具显著区分度。该结果与第 3.6 节相关性分析中“预算、热度与票房正相关、评分与票房弱相关”的结论相互印证。

表 3.4 票房预测 XGBoost 特征重要性 Top 10

排名	特征	相对重要性 (%)
1	budget	48.6
2	tmdb_popularity	22.3
3	movie_year	6.8
4	rating_count	6.1
5	log_budget	6.0
6	runtime_minutes	3.4
7	log_tmdb_popularity	2.4
8	rating_mean	2.3
9	log_rating_count	1.9
10	lang_en	1.9

3.4 电影检索功能实验

电影检索模块支持按标题、简介、类型和标签进行统一关键词搜索，并可通过类型、语言、最低评分等参数组合过滤，同时提供按热度、评分、票房、年份升降序排列。实验中对以下场景进行了验证：

- **关键词搜索：**输入“Star”可召回标题、简介、类型或标签中包含该关键词的 174 部电影，涵盖 *Star Wars* 系列及标题含“star”的其他影片，分页默认每页 25 条；
- **类型过滤：**限定 Sci-Fi 类型后，结果集与 genre_stats.csv 中该类型总量一致；
- **语言过滤：**支持“中文”、“English”等多种别名写法，由后端统一映射至 ISO 语言代码；
- **详情展示：**点击电影条目后，右侧面板展示评分统计、预算票房、简介与标签等完整字段。

实验表明，检索服务能够较快响应本地 CSV 查询请求，满足交互式检索与详情展示需求。该模块为推荐页的种子电影选择与可视化页的样本钻取提供了统一数据入口。

3.5 推荐系统实验

3.5.1 离线产物覆盖

推荐离线构建结果显示：

- 内容推荐 artifacts 覆盖 9727 部电影；
- 协同过滤 artifacts 覆盖 9727 部电影、610 用户、100811 评分；
- 服务健康检查接口返回 artifacts_ready: true。

3.5.2 在线推荐样例

以种子电影“Toy Story”为例，调用统一后端推荐接口：

```
POST /recommendations
{"mode":"content","seedMovieName":"Toy Story","topK":3}
```

系统能够返回包含电影 ID、相似度分数、推荐理由与来源标签的 items 列表，说明 server -> recommender 链路联通正常。

对协同过滤场景，传入 user_id=1 与 movie_name=Toy Story 时，系统能够同时融合 Item KNN 与 User KNN 结果；仅传入用户 ID 时，则返回纯个性化推荐列表。表 3.5 对比了两种推荐路径的 Top 5 结果：内容推荐高度集中于同系列续作（Toy Story 2 相似度 0.947、Toy Story 3 为 0.782），协同过滤则在保留续作的同时引入 Shrek、The Lion King 等同类型高评分作品，体现文本相似与行为相似两种召回逻辑的差异。

表 3.5 以 Toy Story 为种子的推荐结果对比（Top 5）

排名	内容推荐	相似度	协同推荐	分数
1	Toy Story 2	0.947	Toy Story 2	0.865
2	Toy Story 3	0.782	Shrek	0.835
3	The Good Dinosaur	0.494	The Lion King	0.768
4	Turbo	0.488	Monsters, Inc.	0.749
5	Coco	0.480	Twelve Monkeys	0.718

3.5.3 Agent 推荐实验

/agent/recommend 与 agent-ready 推荐模式可接受自然语言偏好，例如“推荐几部高分科幻片，不要恐怖片”。系统流程为：Planner 规则解析偏好 → RAG 检索相关文档与电影字段 → 调用推荐服务生成候选 → 组织最终推荐解释。

3.6 可视化与统计分析实验

离线可视化模块生成 6 张 SVG 图表，主要发现如下：

1. **预算年代趋势：**在 1970–2020 年区间，有预算记录的电影共 5372 部。年均预算从 1970 年的约 639 万美元上升至 2010 年的约 4463 万美元，反映商业制作成本的整体抬升；
2. **语言—票房结构：**在 6235 部非零票房样本中，英语电影 5699 部，中位数票房约 3043 万美元；法语、日语电影样本量分别为 140 部与 98 部，中位数票房约 629 万与 1200 万美元，小语种长尾明显；
3. **预算—票房关系：**散点图覆盖 5158 部同时满足预算不低于 10 万美元、票房不低于 5 万美元的电影，高预算并不保证高票房，存在大量“高预算低票房”离群点；
4. **相关性热力图：**在 5225 部样本上， $\log_{10}$ 变换后票房与预算的 Pearson 相关系数为 0.67，票房与热度为 0.59，票房与评分计数为 0.46，而票房与评分均值相关性仍较弱。

表 3.6 进一步汇总了原始美元空间下的主要变量相关性（ $n = 5230$ ，预算与票房均非零）。可以看到，预算与票房的 Pearson 相关系数达 0.735，热度与票房为 0.487，而评分均值与票房仅 0.057，再次说明口碑与商业表现在本数据集中并非简单线性对应。

表 3.6 主要变量 Pearson 相关系数（ $n = 5230$ ）

变量对	相关系数
预算 $\leftrightarrow$ 票房	0.735
票房 $\leftrightarrow$ 热度	0.487
预算 $\leftrightarrow$ 热度	0.348
评分 $\leftrightarrow$ 热度	0.144
预算 $\leftrightarrow$ 评分	0.072
票房 $\leftrightarrow$ 评分	0.057

前端 Atlas 通过 `/stats/summary`、`/stats/genres`、`/stats/budget-trend`、`/stats/revenue-budget` 与 `/stats/correlations` 在线展示预算趋势、类型分布、预算—票房散点与相关性矩阵等统计视图，其中散点视图按热度选取 Top 160 部影片进行重点展示。

第 4 章 研究结果讨论

4.1 方法有效性分析

本节结合第 3 章实验结果，从数据处理、票房建模、推荐服务与检索统计四个层面，讨论 CineScope 各方法的有效性。

4.1.1 数据处理有效性

ETL 流水线在样本规模与字段质量之间取得了较好平衡。原始电影记录 9742 部，经 TMDb 详情匹配后保留 9727 部，仅丢弃 15 部无法获得有效元数据的影片，保留率达 99.8%。在此基础上，全部终版电影均具有完整 TMDb 标识，财务元数据成功匹配 9386 部，其中 4 部通过 IMDb 回退策略完成补全；非零票房样本 6235 条、非零预算样本 5844 条，为后续回归建模提供了足够训练规模。

从缺失情况看，简介缺失 35 部（0.36%）、片长缺失 41 部（0.42%），关键字段整体可控。评分与标签也随电影主表同步裁剪：终版数据集包含 100811 条评分、3678 条标签，覆盖 610 名用户。该结果说明，基于质量报告的清洗规则能够在不显著牺牲样本量的前提下，为建模与服务层提供稳定、可审计的数据基础。

4.1.2 票房预测有效性

票房预测实验采用 $\log(1 + \text{revenue})$ 作为训练目标，在 6235 条非零票房样本中按 8:2 划分训练集（4988）与验证集（1247），共构建 66 维特征。表 3.3 所示结果表明，XGBoost 与 LightGBM 在 log 空间均取得约 0.64–0.65 的 R^2 ，集成后验证集指标为 $\text{RMSE}_{\log} = 1.275$ 、 $\text{MAE}_{\log} = 0.857$ 、 $R^2_{\log} = 0.652$ 。

进一步比较单模型与集成效果：XGBoost 单独训练时 $R^2_{\log} = 0.652$ ，LightGBM 为 0.641，说明在当前特征体系下 XGBoost 更具优势；按验证集 $\text{RMSE}_{\log}$ 搜索后，集成权重确定为 XGBoost 0.88、LightGBM 0.12，集成模型在 $R^2_{\log}$ 上较 LightGBM 单独提升约 0.011，同时保持与 XGBoost 相当的误差水平。在原始票房空间，集成模型验证集 $\text{RMSE} \approx 9.53 \times 10^7$ 美元、 $\text{MAE} \approx 3.94 \times 10^7$ 美元；由于票房分布高度右偏，log 空间指标更能反映模型对票房数量级的把握能力。

4.1.3 推荐系统有效性

推荐模块离线产物覆盖全部 9727 部电影，协同过滤侧覆盖 610 用户与 100811 条评分；服务健康检查返回 `artifacts_ready: true`，说明向量化器、稀疏矩阵与 KNN 模型均已成功加载。在线实验中，以“Toy Story”为种子调用内容推荐接口，系统能够稳定返回包含电影 ID、相似度分数与来源标签的候选列表，表明 `server` $\rightarrow$ `recommender` 调用链可用。

在协同过滤场景下，同时传入 `user_id=1` 与种子电影时，系统能够融合 Item KNN 与 User KNN 结果；仅传入用户 ID 时，则返回纯个性化列表。以“Toy Story”为例，内容推荐 Top-1 为同系列续作 *Toy Story 2*（相似度 0.947），协同推荐 Top-1 同为该续作（分数 0.865），但协同路径还会引入 *Shrek*（0.835）、*The Lion King*（0.768）等用户群体偏好相近的作品。结合 98.3% 的评分矩阵稀疏度来看，协同过滤能够利用有限行为数据生成候选，而内容推荐则为冷启动与种子影片检索提供了补充路径。两路推荐通过独立微服务部署，也使推荐算法迭代与业务后端聚合实现了有效解耦。

4.1.4 检索与统计有效性

电影检索实验验证了统一搜索与组合过滤策略的可用性。关键词“Star”可召回 174 部相关电影，涵盖 *Star Wars* 系列及标题含“star”的其他影片；限定 Sci-Fi 类型时，结果规模与 `genre_stats.csv` 中记录的 975 部一致；语言别名映射与分页默认值（25 条/页）均按预期工作，说明后端能够在本地 CSV 数据上支撑交互式查询与结果展示。

统计分析结果与特征工程假设相互印证。表 3.6 与表 3.4 共同表明，预算与热度是票房预测的主导信号，而评分均值的重要性相对有限。在 5230 条预算与票房均非零的样本上，预算与票房的 Pearson 相关系数为 0.735，票房与 TMDb 热度的相关系数为 0.487，而票房与评分均值的相关系数仅为 0.057。XGBoost 特征重要性中，`budget` 与 `tmdb_popularity` 合计占比超过 70%，与相关性结论一致。同时，高预算并不必然带来高口碑，与可视化实验中“高预算低票房离群点”的观察相符。前端 Atlas 通过 `/stats/*` 接口在线展示的趋势、散点与相关性视图，与上述定量分析保持一致。

4.2 误差来源与局限性

尽管上述实验支持了方法的有效性，系统仍存在若干不足：

- **样本与特征受限**: 非零票房样本 6235 条, 且未引入导演、演员、档期等外部特征, 限制了预测上限;
- **推荐评估不完整**: 尚未系统计算 Precision@K、NDCG@K 等离线指标, 协同过滤在稀疏场景下仍存在冷启动问题;
- **RAG 覆盖范围有限**: 当前仅索引部分电影画像与摘要文档, 语义召回能力仍有提升空间;

4.3 改进方向

结合上述局限, 后续工作可从以下方向展开:

1. **完善推荐评估**: 补充 Precision@K、NDCG@K 等指标, 并引入更系统的离线评测流程;
2. **增强语义能力**: 采用向量模型替换轻量 RAG, 提高问答与推荐解释质量;
3. **统一混合推荐**: 设计场景化动态加权策略, 并提供单一 Hybrid 接口;

第 5 章 总结与展望

5.1 工作总结

本文围绕电影数据分析与推荐这一典型应用场景，完成了 CineScope 系统的设计、实现与实验验证。研究工作从多源数据清洗出发，构建了包含 9727 部电影、100811 条评分与 3678 条标签的终版数据集，并在保证 99.8% 电影保留率的前提下完成财务元数据匹配与质量审计；在此基础上实现了票房预测、内容推荐、协同过滤推荐、统计可视化、RAG 检索与 Agent 推荐等功能，并通过 React 前端与 FastAPI 后端形成可交互原型。

实验结果表明：

1. ETL 流水线能够在保证样本规模的同时输出可审计的数据质量报告；
2. XGBoost 与 LightGBM 集成模型在验证集上取得 $R_{\log}^2 \approx 0.652$ ，预算与 TMDb 热度为最重要特征；
3. 内容推荐与协同过滤在“Toy Story”等样例上呈现差异化召回，推荐微服务可稳定对外提供 HTTP 接口并支持 Agent 推荐；
4. 三服务架构在联调环境下能够完成端到端演示，具备进一步扩展与二次开发价值。

5.2 主要创新点

CineScope 在工程组织上体现了以下特点：

- 以“数据产物—模型产物—服务接口—前端页面”分层设计，强化模块边界与可维护性；
- 将推荐能力独立为微服务，通过 HTTP 解耦算法迭代与业务聚合；
- 在统一系统中同时覆盖预测、推荐、可视化与问答，形成较完整的数据应用闭环。

5.3 展望

未来工作将聚焦于推荐离线评估体系、语义检索增强、混合推荐策略与容器化部署四个方面。系统建设过程也表明：真实数据清洗与模型训练同等重要，artifact 化管理是在线服务可维护性的基础，推荐系统的体验取决于算法、接口

与交互的协同。总体而言，CineScope 验证了从数据到服务的系统化路径，为后续工程实践提供了可参考的基础框架。